

-0.5cm -0.3cm



Traitement et Analyse de Données

Preprocessing - Cours 2 : Nettoyage des Données

Dr Seydou Nourou SYLLA

Data Scientist Senior
Département TIC
Université Alioune Diop de Bambey

January 25, 2026



Overview

- 1. Détection et suppression des doublons**
- 2. Gestion des données manquantes**
- 3. Gestion des valeurs aberrantes**
- 4. Gestion des incohérences**
- 5. Suppression du bruit**



Détection et suppression des doublons



Pourquoi les doublons posent-ils problème ?

- Répétition non intentionnelle des données.
- Biais dans les analyses statistiques.
- Impact négatif sur les performances des modèles.



Techniques pour gérer les doublons

- **Identification des doublons exacts :**
 - Utilisation des fonctions de hachage ou de tri.
- **Détection des doublons approximatifs :**
 - Mesures de similarité (Jaccard, Cosine).
 - Algorithmes comme Locality Sensitive Hashing (LSH).
- Suppression ou fusion selon le contexte.



Gestion des données manquantes

Pourquoi les données manquent-elles ?

Causes principales

- **Erreurs humaines** : Saisie incomplète, oublis lors de la collecte
- **Problèmes techniques** : Pannes de capteurs, erreurs de transmission, bugs logiciels
- **Restrictions d'accès** : Données confidentielles masquées, permissions insuffisantes
- **Conception du système** : Champs optionnels non remplis, données non applicables
- **Volontaire** : Refus de répondre (enquêtes), données sensibles supprimées

Types de données manquantes

Types de données manquantes

- **MCAR** (Missing Completely At Random) : Manque aléatoire, indépendant des données
 - *Exemple* : Une panne aléatoire d'un capteur de température qui enregistre des valeurs manquantes de manière totalement imprévisible, sans lien avec la température réelle ou d'autres variables
- **MAR** (Missing At Random) : Manque dépendant de variables observées
 - *Exemple* : Dans une enquête, les hommes sont moins susceptibles de répondre à la question sur le salaire. Le manque dépend du genre (variable observée), mais pas du salaire lui-même
- **MNAR** (Missing Not At Random) : Manque dépendant de la valeur manquante elle-même
 - *Exemple* : Les personnes avec un salaire très élevé refusent de déclarer leur revenu. Le manque dépend directement de la valeur du salaire (non observée)

Impact des données manquantes

Problèmes causés

- **Réduction de l'échantillon** : Perte d'information et de puissance statistique
- **Incompatibilité algorithmique** : Beaucoup d'algorithmes ne gèrent pas les valeurs manquantes
- **Performance dégradée** : Modèles moins performants avec données incomplètes

Décision stratégique

Avant de traiter les données manquantes, analyser le **pattern** de manque pour choisir la méthode appropriée.

Stratégies de gestion des données manquantes

Approches principales

1. **Suppression** : Retirer les observations ou variables avec trop de manquants
2. **Imputation simple** : Remplacer par statistiques descriptives (moyenne, médiane, mode)
3. **Imputation avancée** : Utiliser des modèles prédictifs (KNN, régression, etc.)
4. **Imputation multiple** : Générer plusieurs jeux de données imputées
5. **Flags** : Créer des variables indicatrices pour marquer les valeurs manquantes

Critères de choix

- Pourcentage de données manquantes
- Type de variable (numérique vs catégorielle)

Quand supprimer les données ?

Suppression de lignes

- **Critère** : Si une observation a $> 50\%$ de valeurs manquantes
- **Avantage** : Simple, pas de biais d'imputation
- **Inconvénient** : Perte d'information, réduction de l'échantillon
- **Recommandation** : Utiliser si peu de données manquantes ($< 5\%$)

Suppression de colonnes

- **Critère** : Si une variable a $> 70 - 80\%$ de valeurs manquantes
- **Avantage** : Simplifie le dataset
- **Attention** : Vérifier que la variable n'est pas importante pour l'objectif

Imputation par statistiques descriptives

Méthodes

- **Moyenne** : $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$
 - Pour variables continues avec distribution symétrique
 - Sensible aux valeurs aberrantes
- **Médiane** : Valeur centrale après tri
 - Robuste aux valeurs aberrantes
 - Préférable pour distributions asymétriques
- **Mode** : Valeur la plus fréquente
 - Pour variables catégorielles
 - Peut être utilisé pour variables numériques discrètes

Quand l'utiliser ?

Méthode de référence pour données MCAR avec peu de manquants (< 10%).

Imputation par interpolation

Principe

Utiliser les valeurs observées avant et après la valeur manquante pour estimer la valeur.

Techniques

- **Interpolation linéaire :**

$$x_t = x_{t-1} + \frac{x_{t+1} - x_{t-1}}{t_{t+1} - t_{t-1}} \times (t_t - t_{t-1})$$

- **Interpolation polynomiale :** Utilise plusieurs points pour ajuster un polynôme
- **Forward fill / Backward fill :** Répéter la dernière valeur connue
- **Spline interpolation :** Interpolation par splines cubiques

Imputation par K-Nearest Neighbors (KNN)

Principe

Pour chaque observation avec valeur manquante, trouver les k observations les plus similaires (basées sur les variables complètes) et utiliser leurs valeurs pour imputer.

Algorithme

1. Pour chaque observation i avec valeur manquante dans la variable X_j
2. Calculer la distance (Euclidienne, Manhattan) avec toutes les observations complètes
3. Sélectionner les k plus proches voisins
4. Imputer : $x_{ij} = \frac{1}{k} \sum_{l \in \text{voisins}} x_{lj}$ (moyenne) ou médiane

Imputation par régression

Principe

Utiliser un modèle de régression (linéaire, arbre de décision, etc.) pour prédire les valeurs manquantes à partir des autres variables.

Processus itératif

1. **Initialisation** : Imputer par moyenne/médiane
2. **Itération** : Pour chaque variable avec manquants
 - Utiliser les autres variables comme prédicteurs
 - Entraîner un modèle sur les observations complètes
 - Prédire et remplacer les valeurs manquantes
3. **Convergence** : Répéter jusqu'à stabilisation

Imputation multiple (Multiple Imputation)

Principe

Générer m jeux de données imputées (typiquement $m = 5$ à 10) pour quantifier l'incertitude due aux valeurs manquantes.

Algorithme MICE (Multiple Imputation by Chained Equations)

1. **Initialisation** : Imputation simple (moyenne/médiane)
2. **Itération** : Pour chaque variable avec manquants
 - Modéliser la variable manquante avec les autres variables
 - Tirer des valeurs de la distribution prédictive (ajout de bruit)
3. **Répétition** : Générer m jeux de données
4. **Analyse** : Analyser chaque jeu séparément
5. **Pooling** : Combiner les résultats selon la règle de Rubin

Règle de Rubin pour combiner les résultats

$$\bar{Q} = \frac{1}{m} \sum_{i=1}^m Q_i, \quad T = \bar{U} + \left(1 + \frac{1}{m}\right) B$$

où Q_i est l'estimateur du jeu i , $\bar{U}$ la variance moyenne, B la variance entre jeux.

Quand l'utiliser ?

Méthode de référence pour analyses statistiques rigoureuses, surtout avec données MAR.



Comparaison des méthodes d'imputation

Recommandation générale

Commencer par des méthodes simples, puis tester des méthodes avancées si nécessaire. Toujours valider l'impact sur le modèle final.



Gestion des valeurs aberrantes



Définition et impact des valeurs aberrantes

- **Valeur aberrante** : Observation significativement différente des autres.
- Effets :
 - Influence excessive sur les modèles.
 - Mauvaise interprétation des données.



Détection des valeurs aberrantes

- **Méthodes statistiques :**

- Scores Z : Détection basée sur l'écart-type.
- Boxplot : Identification des points au-delà des whiskers.

- **Méthodes basées sur les distances :**

- KNN pour évaluer l'éloignement des points.

- **Modèles avancés :**

- Isolation Forest, DBSCAN.



Traitement des valeurs aberrantes

- **Suppression** : Retirer les points identifiés comme aberrants.
- **Transformation** : Appliquer des transformations log ou racine carrée.
- **Remplacement** : Utiliser des valeurs moyennes ou médianes pour les remplacer.



Gestion des incohérences



Détection des incohérences

- **Incohérences typographiques** : Variations dans les noms ou formats.
- **Données contradictoires** : Valeurs en désaccord avec d'autres données.
- **Valeurs hors domaine** : Données en dehors des plages valides.



Correction des incohérences

- **Uniformisation des formats :**
 - Standardisation des unités, dates et cases.
- **Vérification croisée :**
 - Comparer les données avec des sources de référence.
- **Utilisation de règles métier :**
 - Définir des règles pour valider les valeurs.



Suppression du bruit



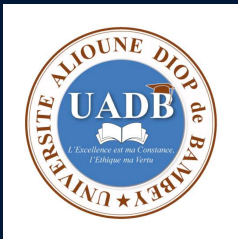
Qu'est-ce que le bruit ?

- **Définition** : Données aléatoires ou non pertinentes masquant les informations utiles.
- Effets :
 - Réduction de la précision des modèles.
 - Complexité accrue dans l'analyse.



Techniques de suppression du bruit

- **Lissage :**
 - Moyenne mobile.
 - Filtrage gaussien.
- **Filtrage :**
 - Suppression des fréquences non pertinentes (FFT).
- **Techniques basées sur le clustering :**
 - Identifier et supprimer les petits clusters considérés comme du bruit.



Thank you for your attention

Dr Seydou Nourou SYLLA

Data Scientist Senior
Departement TIC
Universite Alioune Diop de Bambey

January 25, 2026

Code Python – Détection des doublons

Code Python

```
1 import pandas as pd
2
3 # Détection des doublons exacts
4 duplicates = df.duplicated()
5 print(f"Nombre de doublons: {duplicates.sum()}")
6
7 # Détection sur colonnes spécifiques
8 duplicates_subset = df.duplicated(subset=['id', 'email'])
9 print(f"Doublons sur id et email: {duplicates_subset.sum()}")
10
11 # Voir les doublons
12 df[df.duplicated(keep=False)].sort_values(by=['id', 'email'])
```

Code Python – Suppression des doublons

Code Python

```
1 # Suppression des doublons (garder la première occurrence)
2 df_clean = df.drop_duplicates(keep='first')
3
4 # Suppression sur colonnes spécifiques
5 df_clean = df.drop_duplicates(subset=['id'], keep='first')
6
7 # Suppression en place
8 df.drop_duplicates(inplace=True)
```

Code Python - Suppression des données manquantes (1/2)

Code Python

```
1 import pandas as pd
2 import numpy as np
3
4 # Supprimer les lignes avec au moins une valeur manquante
5 df_clean = df.dropna()
6
7 # Supprimer les lignes où toutes les valeurs sont manquantes
8 df_clean = df.dropna(how='all')
9
10 # Supprimer les colonnes avec valeurs manquantes
11 df_clean = df.dropna(axis=1)
```

Code Python - Suppression des données manquantes (2/2)

Code Python

```
1 # Supprimer les lignes avec plus de 50% de valeurs manquantes
2 threshold = 0.5
3 df_clean = df.dropna(thresh=int(len(df.columns) * (1 - threshold)))
4
5 # Supprimer les colonnes avec plus de 70% de valeurs manquantes
6 missing_pct = df.isnull().sum() / len(df)
7 cols_to_drop = missing_pct[missing_pct > 0.7].index
8 df_clean = df.drop(columns=cols_to_drop)
```

Code Python – Imputation simple (1/2)

Code Python

```
1 import pandas as pd
2 import numpy as np
3
4 # Imputation par la moyenne
5 df['age'].fillna(df['age'].mean(), inplace=True)
6
7 # Imputation par la médiane
8 df['salary'].fillna(df['salary'].median(), inplace=True)
9
10 # Imputation par le mode
11 df['city'].fillna(df['city'].mode()[0], inplace=True)
```

Code Python – Imputation simple (2/2)

Code Python

```
1 # Imputation pour plusieurs colonnes numériques
2 numeric_cols = df.select_dtypes(include=[np.number]).columns
3 for col in numeric_cols:
4     df[col].fillna(df[col].median(), inplace=True)
5
6 # Imputation conditionnelle (selon une autre variable)
7 df.loc[df['gender'] == 'M', 'salary'] = \
8     df.loc[df['gender'] == 'M', 'salary'].fillna(
9         df.loc[df['gender'] == 'M', 'salary'].median()
10    )
```

Code Python – Imputation par interpolation (1/2)

Code Python

```
1 import pandas as pd
2
3 # Forward fill (propagation vers l'avant)
4 df['temperature'].ffill(inplace=True)
5
6 # Backward fill (propagation vers l'arrière)
7 df['temperature'].bfill(inplace=True)
8
9 # Interpolation linéaire
10 df['temperature'].interpolate(method='linear', inplace=True)
```

Code Python – Imputation par interpolation (2/2)

Code Python

```
1 # Interpolation polynomiale
2 df['temperature'].interpolate(method='polynomial', order=2, inplace=True)
3
4 # Interpolation spline
5 df['temperature'].interpolate(method='spline', order=3, inplace=True)
6
7 # Interpolation pour séries temporelles
8 df['date'] = pd.to_datetime(df['date'])
9 df = df.set_index('date')
10 df['value'].interpolate(method='time', inplace=True)
```

Code Python - Détection des valeurs aberrantes (1/2)

Code Python

```
1 import numpy as np
2 from scipy import stats
3
4 # Méthode Z-score
5 z_scores = np.abs(stats.zscore(df['age']))
6 outliers_z = df[z_scores > 3]
7 print(f"Outliers (Z-score > 3): {len(outliers_z)}")
8
9 # Méthode IQR (Interquartile Range)
10 Q1 = df['age'].quantile(0.25)
11 Q3 = df['age'].quantile(0.75)
12 IQR = Q3 - Q1
13 lower_bound = Q1 - 1.5 * IQR
14 upper_bound = Q3 + 1.5 * IQR
15 outliers_iqr = df[(df['age'] < lower_bound) | (df['age'] > upper_bound)]
```

Code Python - Détection des valeurs aberrantes (2/2)

Code Python

```
1 # Suppression des outliers
2 df_clean = df[(df['age'] >= lower_bound) & (df['age'] <= upper_bound)]
3
4 # Remplacement par les limites
5 df['age'] = df['age'].clip(lower=lower_bound, upper=upper_bound)
```

Code Python – Correction des incohérences (1/2)

Code Python

```
1 import pandas as pd
2 from datetime import datetime
3
4 # Uniformisation de la casse
5 df['city'] = df['city'].str.lower()
6 df['city'] = df['city'].str.capitalize()
7
8 # Standardisation des formats de dates
9 df['date'] = pd.to_datetime(df['date'], errors='coerce')
10
11 # Correction des valeurs hors domaine
12 df.loc[df['age'] < 0, 'age'] = np.nan
13 df.loc[df['age'] > 120, 'age'] = np.nan
```

Code Python - Correction des incohérences (2/2)

Code Python

```
1 # Vérification de cohérence logique
2 df['date_achat'] = pd.to_datetime(df['date_achat'])
3 df['date_inscription'] = pd.to_datetime(df['date_inscription'])
4 incoherent = df[df['date_achat'] < df['date_inscription']]
5 print(f"Dates incohérentes: {len(incoherent)}")
6
7 # Correction avec mapping
8 gender_mapping = {'M': 'Male', 'F': 'Female', 'm': 'Male', 'f': 'Female'}
9 df['gender'] = df['gender'].map(gender_mapping).fillna(df['gender'])
```

Code Python – Suppression du bruit (1/2)

Code Python

```
1 import pandas as pd
2 import numpy as np
3 from scipy import signal
4
5 # Moyenne mobile (rolling window)
6 df['temperature_smooth'] = df['temperature'].rolling(window=5, center=True).mean()
7
8 # Filtrage gaussien
9 from scipy.ndimage import gaussian_filter1d
10 df['temperature_smooth'] = gaussian_filter1d(df['temperature'], sigma=2)
```

Code Python – Suppression du bruit (2/2)

Code Python

```
1 # Suppression du bruit avec seuil
2 noise_threshold = df['value'].std() * 2
3 df_clean = df[abs(df['value'] - df['value'].mean()) < noise_threshold]
4
5 # Filtrage par clustering (exemple avec DBSCAN)
6 from sklearn.cluster import DBSCAN
7 clustering = DBSCAN(eps=0.5, min_samples=5).fit(df[['x', 'y']])
8 df['cluster'] = clustering.labels_
9 # Supprimer les points considérés comme bruit (cluster = -1)
10 df_clean = df[df['cluster'] != -1]
```